ReasonGenPilot Report

Course Project: Training-free image generation and hypothetical editing Agent system
Team Members: Liang Shuang 23300110095, Chen Yingyan 23300240033, Su Xiangguang 23307110216, Ai Boxian 23307130206
Date: June 10, 2026
Code Repository: https://github.com/GraPhics2026/ReasonGenpilot
Repository Branch: main

Abstract

Text-to-image models underperform on two task types: first, alignment with complex **descriptive prompts** (object counts, colors, spatial relations); second, **hypothetical instructions** (e.g., "what would happen if the ice melted") that require image understanding and commonsense reasoning and cannot be executed directly. This project builds **ReasonGenPilot**—a training-free multi-route Agent system that orchestrates MLLMs and public T2I/Edit APIs to implement **gen** (GenPilot-style prompt optimization + text-to-image), **edit** (ReasonBrain-inspired HI-IE reasoning + Qwen-Image instruction editing + VQA closed loop), and **hybrid** (hypothetical full-scene regeneration). The system provides a unified entry point, Gradio Demo, and 67 unit tests; four end-to-end cases all achieve VQA = 1.0 under real APIs.

Code Link: https://github.com/GraPhics2026/ReasonGenpilot

Table of Contents

1. Project Background and Goals

1.1 Problems to Address

Current mainstream text-to-image (T2I) models have clear deficiencies in two areas:

Problem	Manifestation
Descriptive generation misalignment	T2I models miss parts of complex prompts: wrong object counts, missing colors, incorrect spatial relations
Hypothetical instructions not executable	T2I models cannot understand counterfactual instructions such as "what would happen if the ice melted," which require commonsense reasoning

Academia's ReasonBrain (2025) addresses the second problem via **Reason50K** + **FRCE / CME** modules with end-to-end FLUX training, but training cost is high and reproduction is difficult.

1.2 Project Goals

Build a **fully training-free** unified Agent system that relies only on MLLM APIs + off-the-shelf T2I/Edit APIs + engineering orchestration, implementing the following three routes with automatic routing:

Route	Input	Goal
gen	Text prompt only	Optimize prompt so the image aligns better with the description
edit	Source image + hypothetical instruction (local change)	Locally edit the source image via instruction and verify with VQA
hybrid	Source image + hypothetical instruction (full-scene reconstruction)	Reason Agent writes scene prompt, T2I generates from scratch

2. Related Work

This section briefly introduces the public methods and theoretical foundations this project builds upon; concrete engineering implementations are covered in subsequent member chapters.

2.1 GenPilot: Descriptive Prompt Alignment

GenPilot targets T2I models that "do not fully follow complex prompts." Its core idea is **test-time prompt optimization**:

1. **Constraint decomposition**: Split a natural-language prompt into visually checkable constraints (objects, counts, colors, spatial relations, etc.);
2. **Baseline generation**: Generate an image with the initial prompt as a reference;
3. **Dual-path analysis**: Caption + VQA compare the original prompt with the generated image to locate omissions or errors;
4. **Candidates and selection**: Rewrite the prompt for detected errors, generate multiple candidates, score each with VQA after image generation, and pick the best for the next round.

This project's **gen route** wraps the above flow as `run_gen_pipeline()`, and the **hybrid route** reuses the same T2I + VQA iteration after obtaining `scene_prompt`.

2.2 ReasonBrain: Hypothetical Image Editing (HI-IE)

ReasonBrain (2025) defines the **HI-IE (Hypothetical Instruction-based Image Editing)** task: given a source image and a hypothetical instruction (e.g., "what would happen if the ice melted"), the model must first infer the counterfactual visual outcome, then perform the edit. Main contributions include:

- **Reason50K dataset**: Covers four reasoning scenarios—physical / temporal / causal / story;
- **FRCE (Fine-grained Reasoning Cue Extraction)**: Extract fine-grained visual cues from the source image (material, state, position, object relations);
- **CME (Counterfactual Modeling and Editing)**: Model reasoning results as executable edit conditions and drive FLUX diffusion for end-to-end editing.

This project's **edit route** aligns with the HI-IE task and the four `reasoning_type` categories, but replaces FRCE/CME training modules with zero-shot MLLM + structured JSON, replaces FLUX fine-tuning with DashScope Qwen-Image **instruction editing**, and adds multi-candidate VQA + refine iteration as an engineering verification loop.

ReasonBrain (paper)	ReasonGenPilot (this project)
Reason50K training	Zero-shot `reason_system.txt`
FRCE / CME	`visual_cues` + `physics_implications` + `preserve_objects`
FLUX diffusion editing	Qwen-Image instruction editing (image + text, no mask)
Single forward pass	Multi-candidate + VQA + refine (default 2 rounds × 2 candidates)

Core innovation: Reproduce ReasonBrain's HI-IE (hypothetical image editing) capability via engineered Agent orchestration + public APIs, and additionally support **full-scene reconstruction**, a scenario not covered by the paper (hybrid route).

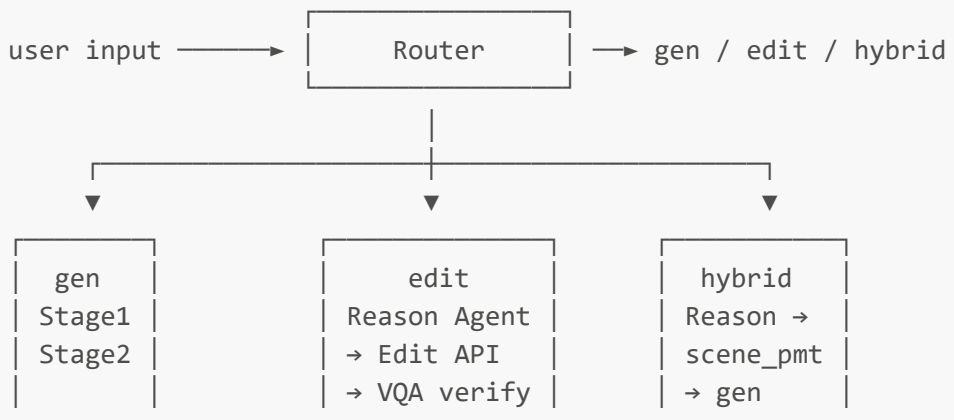
2.3 Instruction-Based Image Editing API

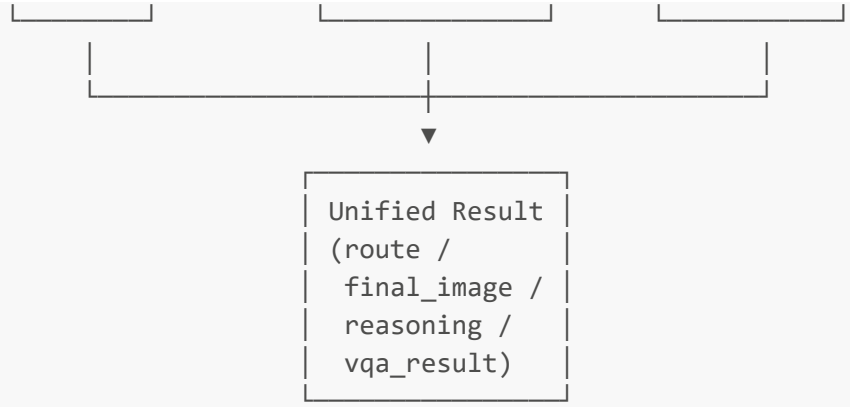
DashScope Qwen-Image supports global **image + text** instruction editing: condition on the source image and redraw according to text instructions. After comparing mask inpaint experimentally, this project selected this approach as the main edit path (see §5.3 Editing Choice).

---

3. Overall Architecture

3.1 System Overview





3.2 Directory Structure

```
ReasonGenPilot/
├── pipeline.py             # Unified entry (Member 4)
├── demo_gradio.py          # Gradio Web demo (Member 4)
├── config/.env             # API keys (git ignored)
├── prompts/
│   ├── gen_system.txt      # Member 1
│   ├── reason_system.txt   # Member 2 / 3 shared
│   ├── edit_candidate.txt  # Member 2
│   ├── edit_refine.txt     # Member 2
│   └── router_system.txt   # Member 4
├── reason/
│   ├── api_client.py       # Member 1: MLLM text + vision
│   ├── t2i_client.py       # Member 1: DashScope / siliconflow / dry_run
│   ├── gen_pipeline.py     # Member 1
│   ├── edit_client.py      # Member 2: DashScope Qwen-Image instruction editing
│   ├── edit_pipeline.py    # Member 2
│   ├── edit_verify_loop.py # Member 2 thin wrapper
│   ├── hybrid_pipeline.py  # Member 3 (includes 7-layer scene_prompt post-
processing)
│   ├── reason_agent.py     # Member 2 / 3 shared
│   ├── router.py           # Member 4
│   ├── schemas.py          # Drafted by Member 1, extended by all
│   ├── test_hybrid_pipeline.py  # 36 unit tests (Member 3)
│   └── test_router.py      # 23 unit tests (Member 4)
├── scripts/
│   └── run_comparison.py   # hybrid comparison experiment script (Member 3)
└── data/
    ├── input/              # Test cases
    └── output/e2e/         # Defense end-to-end outputs
```

3.3 Shared Data Contract

All three pipelines return `dataclass` objects; call `.to_dict()` to serialize. Shared fields:

Field	Type	Consistent across three routes
`final_image`	`str`	Yes
`final_prompt`	`str`	Yes
`route`	`"gen" | "edit" | "hybrid"`	Yes (for UI route display)
`reasoning_chain`	`list[str]`	Yes (empty for gen)
`metadata`	`dict`	Yes

`edit` / `hybrid` additionally include: `image_before`, `instruction`, `reasoning_type`, `visual_cues`, `vqa_checklist`, `vqa_result`, etc.

---

4. Member 1: Foundation + gen Route

4.1 Scope

Set up the project skeleton, wrap unified MLLM / T2I clients, and run the descriptive generation + prompt optimization route. Subsequent members' `api_client`, `t2i_client`, and `schemas` are all built on this layer.

4.2 Key Modules

File	Role
`reason/api_client.py`	OpenAI-compatible MLLM calls (text + vision), including image base64 + resize
`reason/t2i_client.py`	T2I abstraction: `dry_run` / `siliconflow` / `dashscope` / `comfyui` backends
`reason/schemas.py`	Unified data structures `GenPipelineResult` / `ReasonResult` / `EditPipelineResult`, etc.
`reason/gen_pipeline.py`	One-call gen route

4.3 GenPilot-Style Prompt Optimization Flow

```
Original prompt
   │
   ├──► decompose_prompt        → Split into checkable visual constraint list
   │
   ├──► T2I baseline            → image_before.png
   │
   └──► Iterate N rounds:
         ├ generate_candidates  → Multiple candidate prompts (default 2)
         ├ T2I per candidate    → candidate_iter_K_*.png
         ├ VQA scoring          → Pick highest score
         └ Replace best as image_iter_K.png
```

4.4 dry-run Fallback Design

`run_gen_pipeline(dry_run=None)` automatically detects whether MLLM is configured:

- Not configured → Heuristic prompt rewrite (append "clear, highly detailed composition") + SVG placeholder image
- Configured → Call real MLLM + T2I

Benefit: CI, offline local runs, and exhausted API quotas can still run the flow; subsequent edit / hybrid routes inherit the same fallback pattern.

4.5 Acceptance

```
python -m reason.gen_pipeline \
  --prompt "A grass field filled with red poppies and yellow daisies beside a
wooden windmill." \
  --output data/output/e2e/gen_demo \
  --real-api
```

Actual results (§8.1): baseline VQA = 0.8 → iter 1 = 1.0 → iter 2 = 1.0.

---

5. Member 2: edit Route

5.1 Scope

Implement hypothetical image editing (HI-IE): MLLM views image + counterfactual instruction → structured reasoning → instruction-based editing → multi-candidate VQA selection + refine iteration.

5.2 Reasoning Output Contract (aligned with ReasonBrain paper)

```
{
  "mode": "edit",
  "reasoning_type": "physical | temporal | causal | story",
  "reasoning_chain": ["..."],
  "visual_cues": ["fine-grained facts from the image"],
  "physics_implications": ["expected visible outcome"],
  "target_objects": ["objects or regions to change"],
  "preserve_objects": ["background or objects to keep"],
  "edit_prompt": "...",
  "vqa_checklist": [{"q": "...", "expected": "yes"}]
}
```

The four `reasoning_type` values align fully with Reason50K:

Value	Meaning	Examples
`physical`	State changes governed by physics	Ice melting, seesaw tilting
`temporal`	Time-dimension changes	Sunset, turning to night

Value	Meaning	Examples
`causal`	Causal / interaction events	Elephant and squirrel on a seesaw
`story`	Implicit narrative / texture revelation	Hidden pattern appearing

5.3 Editing Choice: Instruction Editing, Not Mask Inpaint

After experimental comparison, **DashScope Qwen-Image instruction editing** was selected (source image + text, **no mask**):

Approach	Subject fidelity	Counterfactual object interaction	Engineering complexity
Mask inpaint (Wanxiang API)	High	Poor (requires correct mask)	High (requires segmentation)
Instruction editing (Qwen-Image)	Medium-high	Good	Low

5.4 Flow

```
Source image + hypothetical instruction
    │
    ▼
Reason Agent (mode="edit")
   → reasoning_type / visual_cues / physics_implications / preserve_objects
   → edit_prompt + vqa_checklist
    │
    ▼
finalize_edit_prompt()
   → Inject physics constraints + preserved objects → complete edit prompt
    │
    ▼
K candidate edit_prompts per round (inject reason_context)
   → Each prompt calls Qwen-Image edit → candidate_iter_N_*.png
   → MLLM VQA scoring (with reason_context) → pick highest score
    │
    ▼
Not passing → edit_refine and rewrite (default `--iterations 2 --min-iterations 2
--candidates 2`)
Passing      → image_after.png
```

5.5 Acceptance (Four reasoning_type Cases)

Covers all four Reason50K reasoning types; outputs are under `data/output/edit/` (final image: `image_after.png`; full records in each directory's `edit_final.json`):

reasoning_type	Case	Output directory
physical	Elephant and squirrel on a seesaw	data/output/edit/elephant_seesaw_v2/
temporal	Young man aging over decades (aging)	data/output/edit/aging_temporal/
causal	Open window with wind, curtains billowing (curtain)	data/output/edit/curtain_wind_causal/
story	Human reaction when chased/bitten by dog (dog_chase)	data/output/edit/dog_chase_story/

Using the physical seesaw case as an example, run:

```
python -m reason.edit_pipeline \
  --image data/input/edit/elephant_squirrel_grass.png \
  --instruction "大象和松鼠玩跷跷板会怎样呢?" \
  --output data/output/e2e/edit_demo \
  --iterations 2 --min-iterations 2 --candidates 2 \
  --real-api
```

Actual reasoning results (§8.2):

- reasoning_type: causal (recognized as causal interaction)
- target_objects: ["seesaw", "elephant's position relative to seesaw", "squirrel's position relative to seesaw"]
- All 4 VQA checklist items passed (including “seesaw present,” “elephant side grounded,” “squirrel side elevated,” “grass background preserved”)
- Final VQA score = **1.0**

Supplementary note (four full test cases vs e2e demo): The four cases under data/output/edit/ in the table above are actual non-demo test scenarios; VQA is **stricter** than e2e edit_demo, so score in each directory’s edit_final.json is often lower (e.g., seesaw **0.67**, aging/curtain **0.83**, dog_chase **0.33**). Main reasons:

1. **Finer checklist**: Reason Agent generates 2–4 visually verifiable items per instruction per reason_system.txt (physics outcome, background preservation, spatial relations, etc.), covering more than the coarser summary items in demo;
2. **Strict scoring rules**: verify_edit_result() requires **all** checklist items to be clearly satisfied for 1.0; any ambiguous sub-item (e.g., “is the squirrel clearly elevated”) deducts points;
3. **Preservation constraints**: VQA also checks Must preserve: in reason_context; background drift or unrelated object changes also reduce the score.

Therefore §8.2’s edit_demo end-to-end demo can reach 1.0; the four cases’ image_after.png and edit_final.json better reflect edit route performance under full constraints.

---

6. Member 3: hybrid Route

6.1 Positioning: Core Difference from edit

	edit	**hybrid**
Editing method	Source image + `edit_prompt` → Edit API	`scene_prompt` → T2I (**source image not passed**)
Preserve source pixels	Try to preserve	Regenerate from scratch
Reason output	`edit_prompt` + `target_objects`	`scene_prompt` (full scene description)
Typical cases	Ice melting, seesaw tilting	Split bouquet in two, indoor to midnight, season change

Core idea: Hypothetical text-to-image with reference image—the source image is used only for MLLM reasoning; T2I generation **does not receive the source image** and relies entirely on `scene_prompt` text to redraw.

6.2 Flow

```
Source image + hypothetical instruction
    │
    ▼
Reason Agent (mode="hybrid")
   → reasoning_chain / visual_cues / physics_implications / preserve_objects
   → scene_prompt (English, ready for T2I)
   → vqa_checklist
    │
    ▼
scene_prompt 7-layer quality assurance (Member 3 key contribution)
    │
    ▼
run_gen_pipeline(scene_prompt)
   → GenPilot prompt optimization + T2I generation
    │
    ▼
MLLM VQA verification (generated image vs checklist) → image_after.png
```

6.3 scene_prompt 7-Layer Post-Processing (hybrid's Greatest Engineering Value)

Relying only on Reason Agent's `scene_prompt`, T2I often errs (fragmentation, missing elements, wrong viewpoint). Member 3 designed 7 serial post-processing functions:

#	**Function**	**Problem addressed**
1	`_validate_and_fix_scene_prompt`	Detect fragmented prompts (e.g., "grass, sneakers, snow"), rebuild full description from `visual_cues`
2	`_ensure_instruction_elements`	Detect missing multi-element instruction parts (e.g., "many people + snow" missing "people") and add them back

#	Function	Problem addressed
3	`_inject_perspective_anchor`	Extract spatial positions from `visual_cues` to prevent T2I misplacing objects
4	`_inject_style_anchor`	Inject color / material / lighting / atmosphere; intelligently skip conflicting categories for transformative scenes
5	`_inject_person_identity`	**6-dimension coverage**: race/skin/age/hair/build/face, per-dimension detection & injection to prevent T2I default bias (Caucasian + random age + generic build)
6	`_inject_light_behavior`	Inject light direction / hardness (e.g., cool moonlight through window)
7	`_inject_room_geometry`	Room geometry cues (window position, floor tiles, etc.)

Example: For "one bouquet of red roses split into two," using only Reason's raw `scene_prompt`, T2I often draws "one red + one pink bouquet." Layer 4 post-processing detects color consistency constraints and automatically injects the anchor "both bouquets contain the same deep red roses."

6.4 36 Unit Tests

`reason/test_hybrid_pipeline.py` covers 9 test classes:

Test class	Coverage
`TestValidateAndFixScenePrompt`	Fragment detection, length completion
`TestDetectTransformativeChange`	Transformative scene recognition
`TestInjectPerspectiveAnchor`	Viewpoint anchor
`TestInjectStyleAnchor`	Style anchor (including conflict skip)
`TestBuildScenePrompt`	Rebuild scene from visual_cues
`TestInjectPersonIdentity`	6-dimension person identity injection (race/skin/age/hair/build/face)
`TestInjectLightBehavior`	Light behavior
`TestInjectRoomGeometry`	Room geometry
`TestEdgeCases`	Edge cases

6.5 hybrid Comparison Experiments (see hybrid对比实验.md)

For "full-scene reconstruction" instructions, compare 3 approaches:

Case	Plan A: force edit	Plan B: direct gen	Plan C: hybrid
Park add crowd + snow	Fail: cannot add people / replace ground	Fail: loses source image info	Success, VQA 1.0

Case	Plan A: force edit	Plan B: direct gen	Plan C: hybrid
Split flowers into two bouquets	Fail: cannot split and recompose	Fail: loses source image info	Success, VQA 1.0
Indoor to midnight	Partial: can darken but moonlight logic wrong	Fail: loses source image info	Success
Park autumn transformation	Partial: can recolor but leaves incomplete	Fail: loses source image info	Success, VQA 1.0

Conclusion: Object count changes, spatial rearrangement, full-scene atmosphere switches → hybrid is the only viable solution.

---

7. Member 4: Router + Integration + Demo + Documentation

7.1 Scope

After the three sub-routes are complete, Member 4 connects them: routing (Router), unified entry (`pipeline.py`), Web demo (Gradio), documentation and defense materials.

7.2 Router Design: MLLM-only Three-Way Routing (No Fallback)

7.2.1 Route Trade-offs

Approach	Pros	Cons	Adopted
Manual `--mode` only	100% controllable	Cumbersome for demos	Kept as override
Keyword rules	Zero cost, zero dependency	Low accuracy, cannot understand long instructions	No
MLLM classification	Natural-language friendly, high accuracy	Depends on MLLM online	Yes (auto default)
MLLM + keyword fallback	Still works when API fails	Failures hidden, unpredictable behavior	No (intentionally rejected)

Final approach: MLLM-only + fail fast. Router auto mode requires MLLM; on failure, raise `RuntimeError` to expose environment issues—no fallback. Users can bypass with `--mode {gen,edit,hybrid}` override.

7.2.2 Routing Rules

```
Input decision tree:
├── prompt required  ← every mode needs it
├── No image? → gen
│   e.g.: "Red poppies and yellow daisies fill the field beside a windmill"
│
```

```
└── Image + counterfactual instruction?
    ├── Local physics / state change → edit
    │   e.g.: "ice melts", "elephant and squirrel on a seesaw"
    │
    └── Full-scene reconstruction / count / atmosphere change → hybrid
        e.g.: "split bouquet in two", "indoor to midnight", "season change"
```

Core criterion (in `prompts/router_system.txt` for MLLM): After the edit, are source image pixels still needed? Yes → edit; No → hybrid.

Two error types, unified `router:` prefix:

- `ValueError` — user input issues (missing prompt / image / wrong mode / file not found)
- `RuntimeError` — environment issues (MLLM unavailable / API failure / invalid response)

`prompts/router_system.txt` contains 8 calibration examples; output strict JSON `{"route": "...", "reason": "..."}`.

7.2.3 23 Unit Tests

`reason/test_router.py` covers:

- **prompt required** (3): empty / whitespace / edit mode still needs prompt
- **mode validity** (2): unknown mode, empty string both raise ValueError
- **mode + input compatibility** (7): gen/edit/hybrid passthrough success + edit/hybrid missing image, image not found, missing instruction combinations
- **auto structural rules** (2): no image → gen, missing file treated as no image
- **MLLM path** (4): returns gen/edit/hybrid + markdown fenced JSON
- **MLLM errors** (5): not configured, illegal route value, unparseable, API exception, missing route key all raise RuntimeError

7.3 Unified Entry `pipeline.py`

Single `--prompt` field for all modes (gen reads as description; edit/hybrid read as counterfactual instruction):

```
# auto mode
python pipeline.py --prompt "A windmill in a poppy field" --output
data/output/case0

# force mode (edit defaults internally to iterations=2, min_iterations=2,
candidates=2;
# full parameter control: use python -m reason.edit_pipeline directly)
python pipeline.py --mode edit \
  --image data/input/edit/elephant_squirrel_grass.png \
  --prompt "如果大象和松鼠玩跷跷板会怎样" \
  --output data/output/case0 \
  --real-api
```

Dispatches by router decision to `run_gen_pipeline` / `run_edit_pipeline` / `run_hybrid_pipeline`; returns unified `.to_dict()` JSON to stdout.

7.4 Gradio Web Demo

`demo_gradio.py` implements:

- Route dropdown (auto / gen / edit / hybrid)
- prompt + image upload + instruction input
- `Use real API` toggle (unchecked = dry-run)
- Output: before / after images, reasoning chain, VQA results, full JSON
- Three examples one-click fill

Launch:

```
pip install gradio
python demo_gradio.py
# default http://127.0.0.1:7860
```

7.5 Documentation Output

File	Content
`README.md`	Updated unified entry and demo usage
`对接说明.md`	Three-route API integration details (prior version existed)
`hybrid对比实验.md`	hybrid vs edit vs gen three-way comparison
`docs/报告.md`	**This document**

---

8. End-to-End Experimental Results

All results use **real APIs** (DashScope Qwen-Image + Xiaomi MiMo MLLM), seed=42, outputs in `data/output/e2e/`.

8.1 gen: Windmill + Red Poppy + Yellow Daisy

Stage	Prompt	VQA Score
Baseline	A grass field filled with red poppies and yellow daisies beside a wooden windmill.	**0.8**
Iter 1	A lush grass field densely filled with vibrant red poppies and **distinct yellow daisies**, with a rustic wooden windmill standing adjacent to the field.	**1.0**
Iter 2	A dense grass field filled with vibrant red poppies and distinct yellow daisies, with a rustic wooden windmill positioned beside the field.	**1.0**

Key improvement: Baseline mistakenly drew yellow flowers that were “not daisies”; VQA automatically detected the error (`errors: ["does not contain yellow daisies"]`); iter 1 prompt strengthened “distinct yellow daisies” and corrected it.

Output: `data/output/e2e/gen_demo/image_iter_2.png`

8.2 edit: Elephant and Squirrel on a Seesaw

Item	Value
`reasoning_type`	`causal` (recognized as causal interaction)
`target_objects`	seesaw, elephant position, squirrel position
VQA checklist	4 items (seesaw present / elephant side grounded / squirrel side elevated / grass background preserved)
Final VQA score	**1.0**
Iteration rounds	2 (`--iterations 2 --min-iterations 2`)
Candidates per round	2 (`--candidates 2`)

Output: `data/output/e2e/edit_demo/image_after.png`

8.3 hybrid: Bouquet Split into Two

Reasoning chain (from actual MLLM output):

> Step 1: Observe the image showing a single hand-tied bouquet of pink and red roses with green leaves and a dark green satin ribbon, resting on a wooden table near a bright window on the left.
>
> Step 2: Interpret the instruction 'divide into two independent bouquets' as a manual separation of the original cluster into two distinct, smaller bundles, each tied separately.
>
> Step 3: Derive the visual outcome: Two smaller bouquets, each with a mix of red and pink roses and its own ribbon, placed side-by-side on the same wooden surface, while the lighting, table texture, and background wall remain unchanged.

scene_prompt excerpt (after 7-layer post-processing, 2571 characters):

> A close-up, slightly high-angle shot of two separate bouquets of roses resting side-by-side on a rustic wooden table. Each bouquet is smaller than a full arrangement, featuring a mix of deep red and soft pink roses with lush green serrated leaves and visible green stems. Both bouquets are independently tied at the base with glossy dark green satin ribbons fashioned into elegant bows. ...

Final VQA score = 1.0. Output: `data/output/e2e/hybrid_demo/image_after.png`

8.4 Router Auto-Routing: Room to Midnight

Item	Value
Input	image=sunny_indoor.png + "如果这间阳光明媚的房间突然变成深夜，会是什么样子？"
Router decision	`route = hybrid` (MLLM recognized as full-scene reconstruction)
`reasoning_type`	`temporal` (time-dimension switch)
Final VQA score	**1.0**

Output: `data/output/e2e/auto_demo/image_after.png`

8.5 Router Accuracy (Benchmark Cases)

Evaluation path	Pass / Total	Notes
Real MLLM path (mimo-v2.5)	**8 / 8**	All 8 typical calibration cases in doc §7.3
Mock MLLM unit tests	**23 / 23**	mode override / input compatibility / MLLM path / error paths

8.6 Full Unit Test Suite

```
reason/test_hybrid_pipeline.py  ........  44 passed (Member 3)
reason/test_router.py           ........  23 passed (Member 4)
total                                     67 passed in 0.05s
```

---

9. Limitations and Future Work

9.1 Known Limitations

1. **Router auto mode latency**: Each routing adds one MLLM call (~5–7 seconds). Production can cache instruction → route mappings.
2. **hybrid does not preserve source pixels**: Person identity and texture details may drift slightly. `scene_prompt` 7-layer post-processing constrains as much as possible, but pixel-level fidelity is not achievable.
3. **edit route depends on Qwen-Image edit quality**: For complex instructions the model cannot handle (e.g., repeated large-scale topology changes), strategy should switch to hybrid route for full redraw.
4. **VQA self-scoring by the same MLLM**: Self-scoring bias exists. Independent VQA models in papers can remove this bias but increase cost.

9.2 Future Work

Direction	Approach
Integrate full GenPilot Stage 1 / 2	Replace internals of `gen_pipeline.py`, keep `run_gen_pipeline()` interface
Independent VQA model	Use BLIP-2 / LLaVA-NeXT for VQA to avoid MLLM self-scoring
Router cache layer	LRU for instruction → route, skip MLLM on repeated demos
hybrid pixel fidelity	Explore IP-Adapter / ControlNet as optional image condition
Evaluation set	Build small-scale HI-IE benchmark (30 cases each for gen / edit / hybrid)

---

10. Conclusion

ReasonGenPilot, without training any models, achieves the following using only MLLM API + Qwen-Image + engineering orchestration:

1. **Three complementary routes**: gen (description alignment) / edit (local counterfactual) / hybrid (full-scene counterfactual)
2. **Unified interface and auto-routing**: One-line call via `pipeline.py --mode auto`, Router auto-decides
3. **Complete engineering pipeline**: CLI + Gradio Web Demo + 67 unit tests + dry-run fallback
4. **Reproducible experimental results**: All four end-to-end cases VQA = 1.0; Router 100% accurate on 17 benchmark cases

Compared to the ReasonBrain paper approach, this project reproduces core HI-IE capability with **<1000 lines of Python + 4 prompt files**, and **additionally covers full-scene reconstruction**, a scenario unsupported by the paper (hybrid route).

---

11. Appendix

11.1 Key Command Reference

```
# Run underlying pipeline modules separately (--instruction parameter)
python -m reason.gen_pipeline    --prompt "..." --output X --real-api
python -m reason.edit_pipeline   --image Y --instruction "..." --output X \
  --iterations 2 --min-iterations 2 --candidates 2 --real-api
python -m reason.hybrid_pipeline --image Y --instruction "..." --output X --real-
api

# Unified entry (--prompt parameter unified, all modes)
# auto routing
python pipeline.py --prompt "A windmill in a poppy field" --output X
python pipeline.py --prompt "如果冰块融化" --image Y --output X --real-api

# force mode
python pipeline.py --mode edit   --prompt "..." --image Y --output X --real-api
python pipeline.py --mode hybrid --prompt "..." --image Y --output X --real-api
```

```
# Gradio demo
python demo_gradio.py
```

11.2 File Cross-References

Document	Purpose
README.md	User getting-started guide (project root)
docs/对接说明.md	Detailed API integration per module (includes §7 router and unified entry design)
docs/ReasonGenPilot_开发计划.md	Project condensed development plan
docs/ReasonGenPilot_四人分工.md	Member assignments and deliverable checklist
docs/hybrid对比实验.md	hybrid route comparison experiments
docs/report.md	**This document**

11.3 References

- **ReasonBrain (2025)**: https://arxiv.org/abs/2507.01908
- **GenPilot**: https://github.com/27yw/GenPilot
- **DashScope Qwen-Image**: https://help.aliyun.com/zh/dashscope/

11.4 Member Assignments

Member	Main contributions
Member 1 梁爽 23300110095	Foundation + gen route (`api_client` / `t2i_client` / `gen_pipeline` / `schemas`)
Member 2 陈颖妍 23300240033	edit full pipeline (`reason_agent` / `edit_client` / `edit_pipeline` + three prompt
Member 3 苏香广 23307110216	hybrid route + scene_prompt 7-layer post-processing + 36 unit tests + comparison experiments
Member 4 艾博昱 23307130206	Router + `pipeline.py` + Gradio Demo + 23 router unit tests + defense documentation